

Task 2 — QA Strategy & Thinking

Scenario: Day one as the first QA Engineer at a fintech startup. A mobile trading app (iOS + Android) is two weeks from its first public release. No existing test suite, no QA documentation, dev team has been shipping fast, and real user funds are involved.

1. Where do you start?

Before writing a single test, I'd spend day one building a map, not a test plan:

- **Read the product, not just the code.** Install the app on real devices, open an account in a sandbox/staging environment if one exists, and walk every flow a user can walk: onboarding/KYC, deposit, place a trade, withdraw, view history, log out. I want to feel where it's fragile before I formalize anything.
- **Inventory what already exists.** Crash reporting (Crashlytics/Sentry), analytics, CI pipeline, staging environments, feature flags, existing manual QA checklists (even a Notion doc counts), API docs/Postman collections, and the backlog/ticket history for "known issues." Fast-shipping teams almost always have *some* institutional knowledge, it's just not written down as "QA docs" — it lives in Slack threads and senior engineers' heads.
- **Find out what "money-safe" already means here.** Ask directly: is this connected to real payment rails yet, or is launch day still using a test ledger/sandbox exchange? This single answer changes the entire risk profile.
- **Identify the two or three riskiest flows** — for a trading app that's almost always: funding/deposits, trade execution, and withdrawals — and get eyes on those first, manually, today. I'm not trying to have full coverage on day one; I'm trying to make sure the worst possible bug (user loses money, or money appears/disappears incorrectly) isn't sitting in the code path that ships in two weeks.
- **Talk to the team**, not just the tickets. Ask engineering what *they're* nervous about. The person who wrote the order-matching or wallet-balance code usually already knows where the edge cases are.

2. How would you approach testing this app?

With two weeks and zero existing coverage, I can't build exhaustive automation before launch — I have to triage by **risk x likelihood**, and split effort between "stop the bleeding" and "build the foundation."

Risk-based prioritization, roughly in this order:

1. **Money movement correctness** — deposits, withdrawals, balance updates, trade execution/settlement, fee calculation. Any bug here is a legal/financial incident, not a UX bug.
2. **Auth & account security** — session handling, biometric/PIN lock, logout-on-backgrounding, token expiry, account lockout. A trading app is a target.
3. **Core trading UX** — order placement, order book/price updates, order cancellation, error states when an order fails or a market is closed.
4. **Platform parity** — iOS and Android frequently diverge on things like biometrics, deep links, push notifications, and keyboard/numeric input handling for amounts. I'd explicitly test both, not assume symmetry.
5. **Network resilience** — trading apps live on mobile networks: flaky connectivity, backgrounding mid-transaction, app killed during a pending order, retry/idempotency behavior. "What happens if the user's train goes through a tunnel while an order is in flight" is a real, common bug source.
6. **Everything else** (settings, notifications, marketing screens) — lowest priority for manual/automated depth pre-launch.

Test mix:

- **Manual exploratory testing** on real devices (not just simulators) for the top-risk flows, done by me personally in week one — this finds more real bugs per hour than anything else at this stage.
- **API/contract tests** against the backend directly (not through the UI) for balance, trade, and withdrawal endpoints — fastest signal, catches business-logic bugs before UI even matters, and I can start writing these on day one in parallel with exploratory testing.
- **A thin layer of E2E UI automation** (Appium/Detox/Maestro depending on stack) on the 5-10 flows that must never break: login, deposit happens (sandbox), place trade, withdraw, view balance. Thin and reliable beats broad and flaky with two weeks on the clock.

- **Negative and edge-case testing as a first-class citizen**, not an afterthought: invalid amounts, decimal/precision edge cases, double-tap on "submit" (idempotency), insufficient balance, expired session mid-trade, market closed, price slippage between confirm and execute.
- **Non-functional passes**: basic security review (is anything sensitive logged? is API traffic pinned/TLS-enforced? are tokens stored securely, not in plain SharedPreferences/UserDefaults?), and a performance/battery sanity check.

I would *not* try to hit a coverage percentage. I'd rather have 20 rock-solid, high-value automated checks running in CI than 200 flaky ones that get muted within a month.

3. What does QA look like inside a sprint, from ticket creation through to regression?

- **Ticket creation**: QA is looped in when a ticket is written, not when it's "done." I push for acceptance criteria to be written *on the ticket* before dev starts — what does success look like, what are the known edge cases, is money involved. For anything touching balances/trades/payments, I add a short "risk note" directly on the ticket.
- **During development**: I don't wait for a build. I review the PR description and, where useful, the diff itself for obvious risk (missing validation, unhandled error paths, no idempotency key on a payment call). I test against a dev/staging build as soon as one exists, not just at the end of the sprint.
- **Before merge**: the PR must include or update automated test coverage for the behavior it changes — CI runs unit + relevant API/E2E tests on every PR, and I treat "no tests because it was urgent" as a flag to raise, not a rule to accept.
- **QA pass**: manual verification of the acceptance criteria plus a short exploratory pass around the change (adjacent flows, edge cases the ticket didn't mention). Bugs go back with clear repro steps, device/OS info, logs, and severity.
- **Regression**: every merge to the release branch triggers the automated regression suite (see below) against a staging build. I maintain a small, prioritized *manual* regression checklist for the flows automation doesn't yet cover, and run it before any release candidate is cut.
- **Release gate**: nothing goes to the app stores without: automated suite green, manual smoke pass on real iOS + Android devices, and an explicit sign-off against the release readiness checklist (crash-free rate, no open Sev1/Sev2, rollback plan confirmed).
- **Post-release**: I treat production monitoring (crash rate, error rate on payment endpoints, support tickets) as an extension of the regression loop — the first 48 hours after a fintech release get active dashboard watching, not "check back next sprint."

4. What does your ideal regression suite look like?

Structured as a pyramid, weighted toward speed and determinism:

- **Unit tests (widest layer)**: business logic in isolation — fee calculation, balance math, precision/rounding rules, order validation logic. Fast, deterministic, run on every commit.
- **API/contract tests (large layer)**: hit the backend directly for every money-movement and trading endpoint — happy path plus the negative cases (insufficient funds, invalid amount, expired token, duplicate/idempotent requests, closed market). These catch the majority of real fintech bugs and run in minutes.
- **UI E2E — critical path only (thin top layer)**: a small, curated set of true end-to-end mobile flows on real devices/emulators via cloud device farms (BrowserStack/Sauce Labs/Firebase Test Lab) covering: onboarding → fund (sandbox) → trade → withdraw → view history, run on both iOS and Android, on at least one older OS version and one current one.
- **Cross-cutting, run on a schedule rather than every commit**:
 - Security regression: auth bypass attempts, token expiry, sensitive-data-in-logs checks.
 - Performance/load on trade-execution and price-feed endpoints.
 - Visual regression on key screens (balance, order ticket) to catch layout breaks that hide real numbers.
- **Test data**: a dedicated sandbox environment with a reset-able seeded ledger, never production financial systems, and synthetic accounts only — this is non-negotiable for a suite that touches money logic.
- **Ownership**: every suite has an owner, a documented flake-quarantine process (a flaky test gets fixed or removed within a sprint, never just re-run until green), and a dashboard the whole team can see — regression is worthless if it's a black box only QA looks at.

5. What would keep you up at night about this app specifically, and releasing to the public?

- **Money correctness under concurrency and partial failure:** two rapid taps on "confirm trade," a network retry that resubmits an order, an app killed mid-withdrawal, a price changing between confirmation and execution. If the backend isn't idempotent and the ledger isn't reconciled against an external source of truth, these produce real financial discrepancies, and they're exactly the bugs that never show up in a single manual pass.
- **No existing test suite two weeks from launch.** That means the team's confidence in "it works" is based on manual spot-checks and developer intuition. I'd worry most about the parts of the app *nobody* has deliberately tried to break yet — because with fast shipping, that's usually most of it.
- **iOS/Android divergence on security-sensitive behavior** — biometric auth, secure storage, background/foreground session handling — being tested on one platform and assumed on the other.
- **Regulatory/compliance exposure:** KYC/AML flows, data residency, and whether the "two weeks to public release" timeline has actually accounted for any compliance sign-off, or whether QA is the only gate standing between fast shipping and a regulatory problem.
- **No rollback/kill-switch plan.** If a critical money-bug ships, can the team disable trading or withdrawals remotely (feature flag) without an app store resubmission that takes days? If not, that's the single biggest risk to close before launch, bigger than any individual test I could write.
- **Silent data loss or drift between the app's displayed balance and the backend's actual ledger** — a UI bug here doesn't look scary, but it directly erodes user trust in a way that's very hard to win back in fintech.

In short: the tests matter, but the thing that actually keeps me up at night is whether the *organization* has a safety net (rollback, monitoring, idempotent backend design, a sandboxed test environment) around the tests — because two weeks of QA cannot fully compensate for the absence of that.